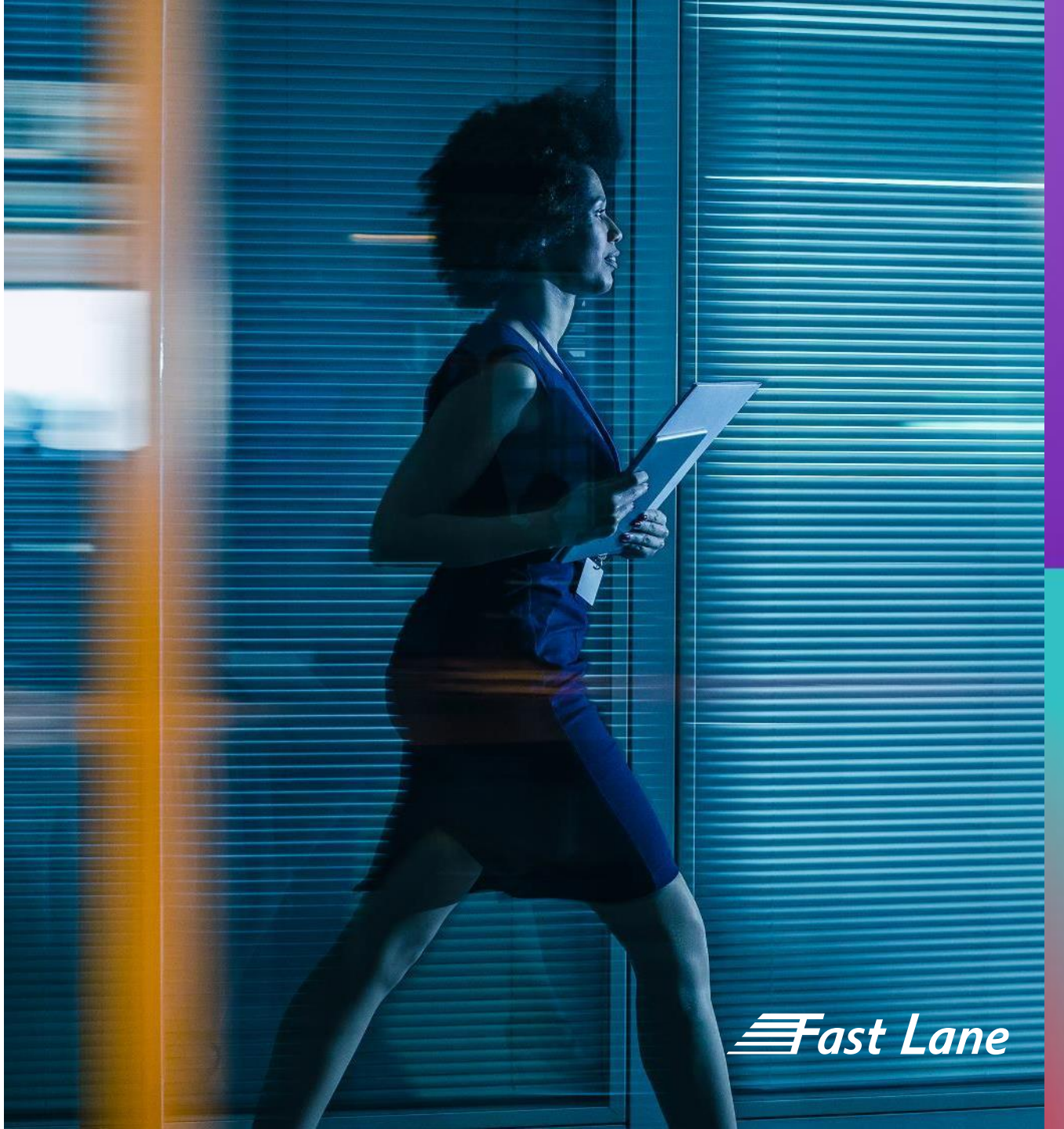
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 9

Agentic DevOps & Automation

A deployment agent, an Ubuntu host and nothing but SSH

Agentic Software Engineering with Claude Code



01

The Deployment Agent

Allowlist, manifest and the proof contract

02

SSH Deployment

From a fresh Ubuntu box to HTTPS

03

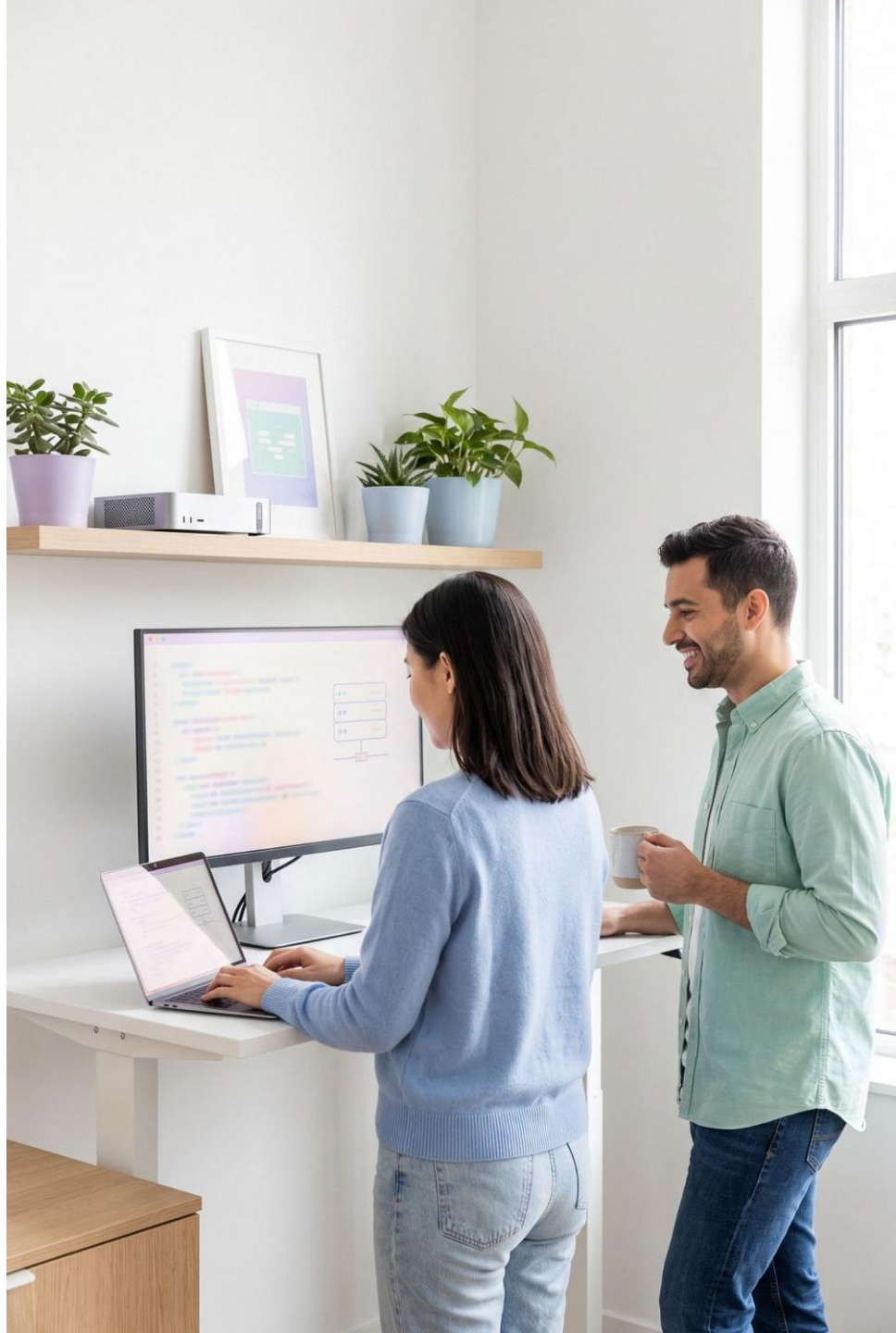
Web Vitals and SEO

Lighthouse CI and the agentic-seo fix loop

Agentic DevOps

A deployment agent, an Ubuntu host and nothing but SSH

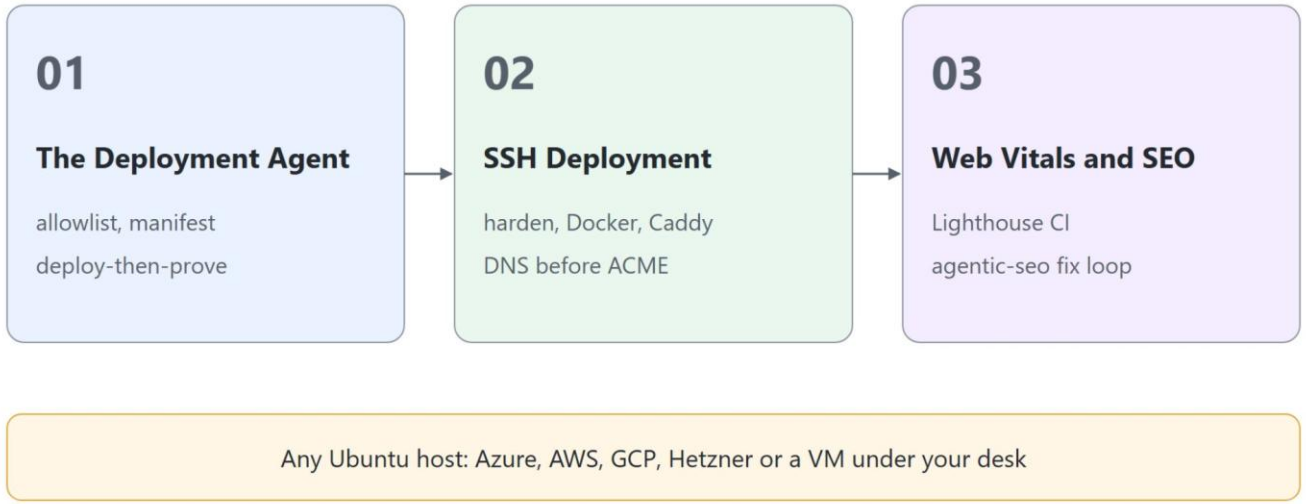
- **The agent**: a deployment sub-agent with a scoped allowlist, a manifest and a proof contract
- **The host**: a plain Ubuntu box reached over SSH, hardened, with Docker and a Caddy edge
- **The site**: Core Web Vitals and SEO checked after every deploy



Overview

The path through agentic DevOps

- Build the **deployment agent** first: its allowlist, manifest and proof contract
- Take a fresh Ubuntu box to HTTPS in **six ordered phases**
- Keep the deployed site fast with **Lighthouse CI** and the **agentic-seo** skill
- SSH removes the safety nets, so every guarantee becomes a step the agent proves



01

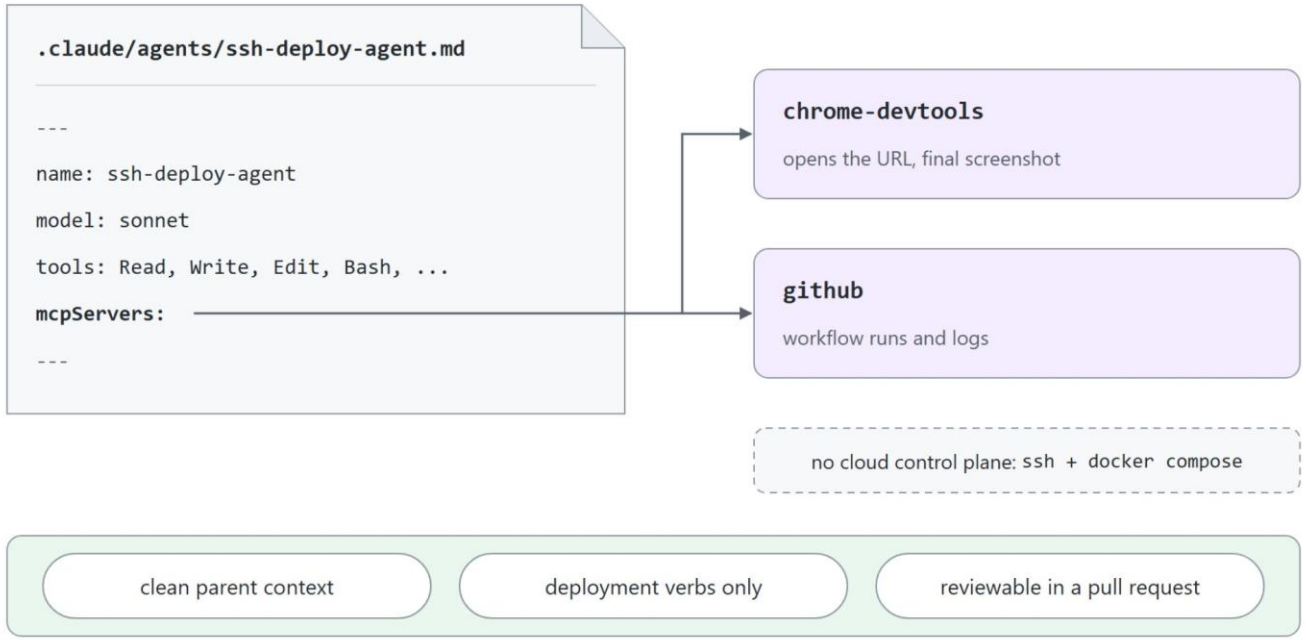
The Deployment Agent

Allowlist, manifest and the proof contract

The deployment agent

A sub-agent that moves a build onto a host and proves it landed

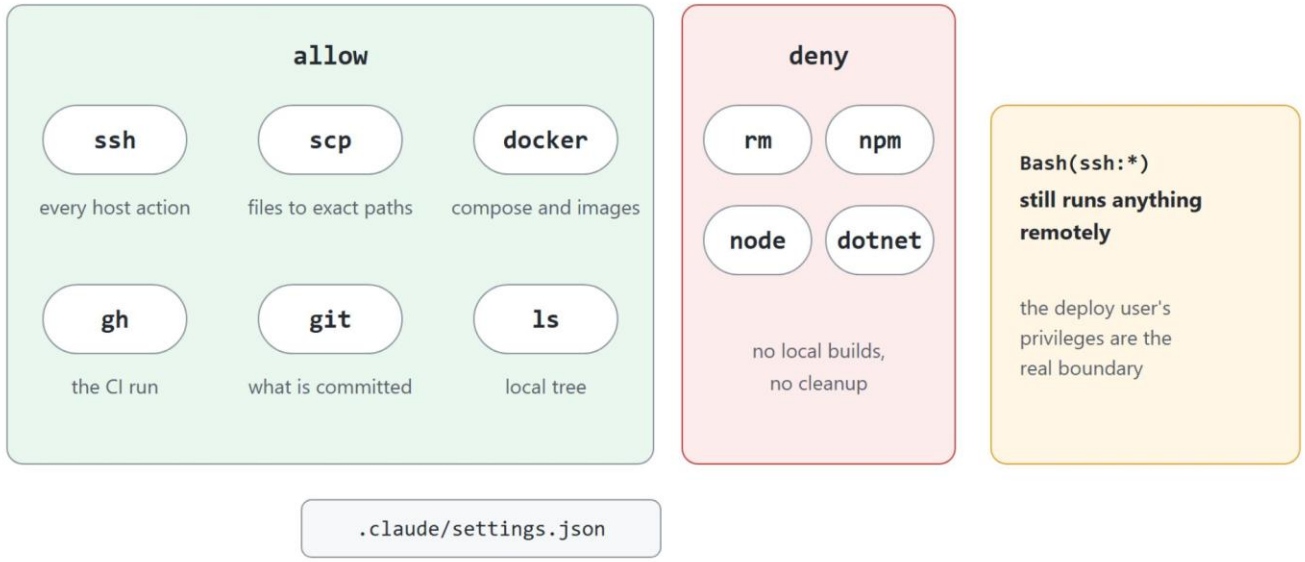
- Deployment runs in its own **sub-agent**, away from application work
- SSH transcripts and log tails stay out of the parent context
- Two MCP servers: **chrome-devtools** for the screenshot, **github** for CI runs
- No cloud MCP: on this target the host is `ssh` and the app is `docker compose`
- The agent is a text file, so its scope is a reviewable diff



The shell allowlist

Six verbs a deployment needs, nothing else

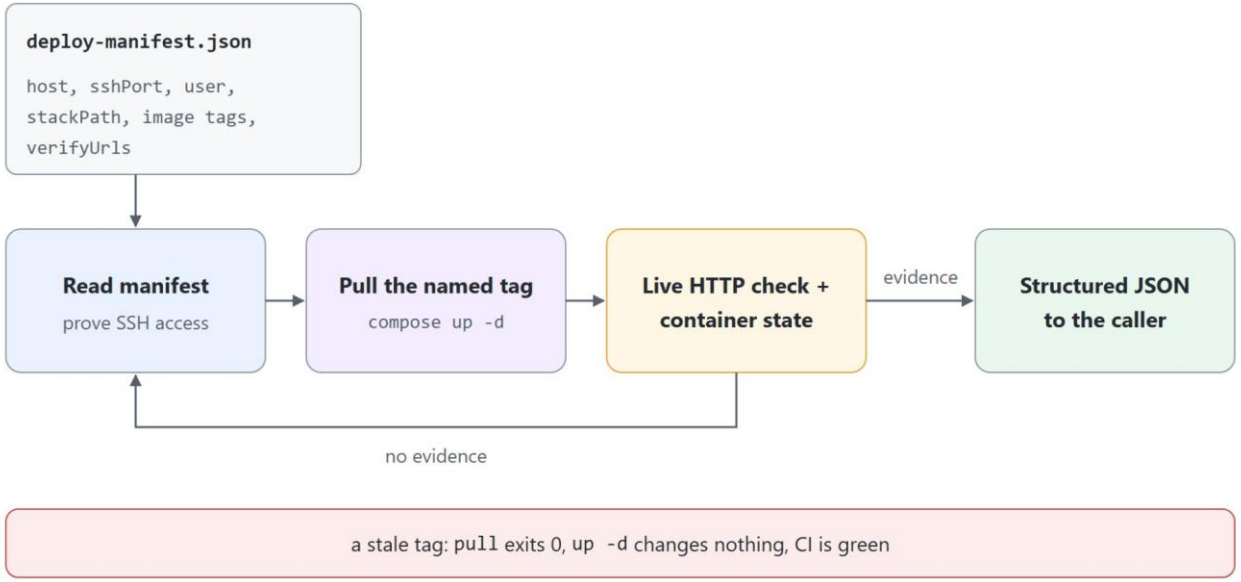
- Command scoping lives in `.claude/settings.json`, not in the agent file
- Allow `ssh`, `scp`, `docker`, `gh`, `git` and `ls`
- Deny `npm`, `node` and `dotnet`: images come from CI, never a local build
- Deny `rm`: no confident cleanup of somebody else's volume
- `Bash(ssh:*)` runs anything remotely, so scope the **deploy user** too



The manifest and the deploy-then-prove loop

Success is an observation, not an assertion

- A committed **manifest** holds host, port, user, path and image tags
- A value missing from the manifest is asked for, never inferred
- Three phases: prove access, pull and recreate, then produce **evidence**
- Two proofs: a live HTTP 200 and the container's image matching the tag
- The result returns as **JSON** with `verifiedUrls` and `containerState`



Stop and ask

When the state is destructive or ambiguous

- A container name or host port is already taken by another service
- A container is **unhealthy** before the deploy starts
- The key authenticates as a different user than the manifest names
- SSH hardening without a verified **second session**
- Cleanup that would remove a volume, an image or a credential file



02

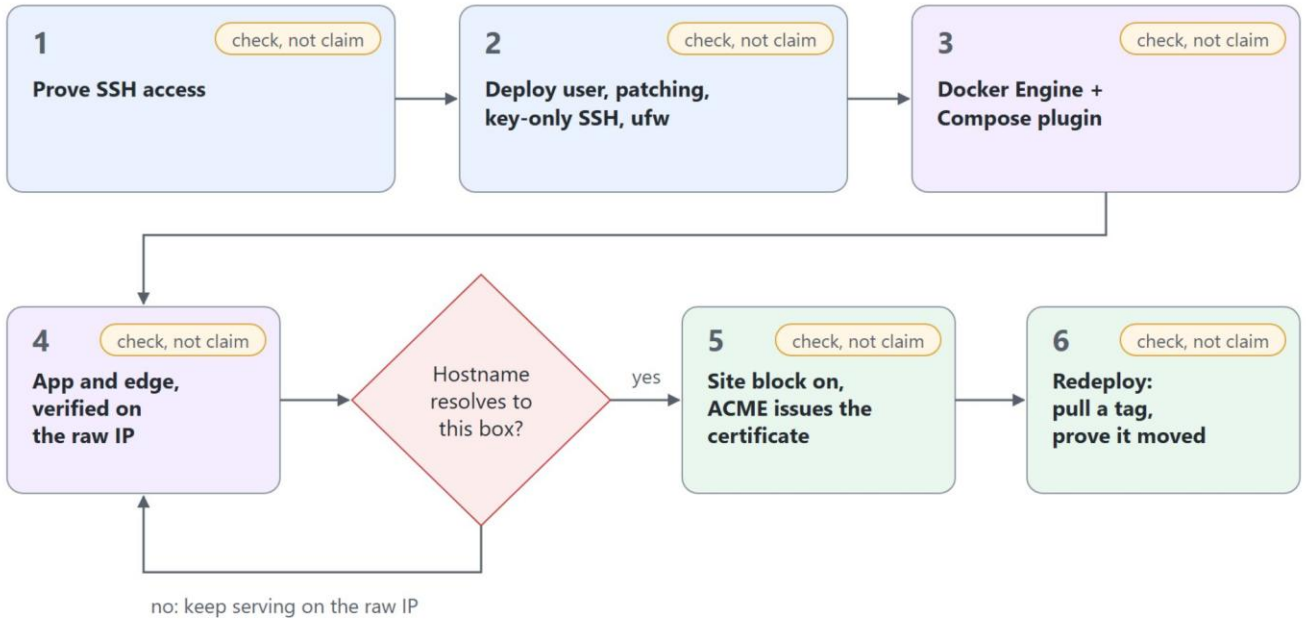
SSH Deployment

From a fresh Ubuntu box to HTTPS

Six phases and one gate

The order is load bearing

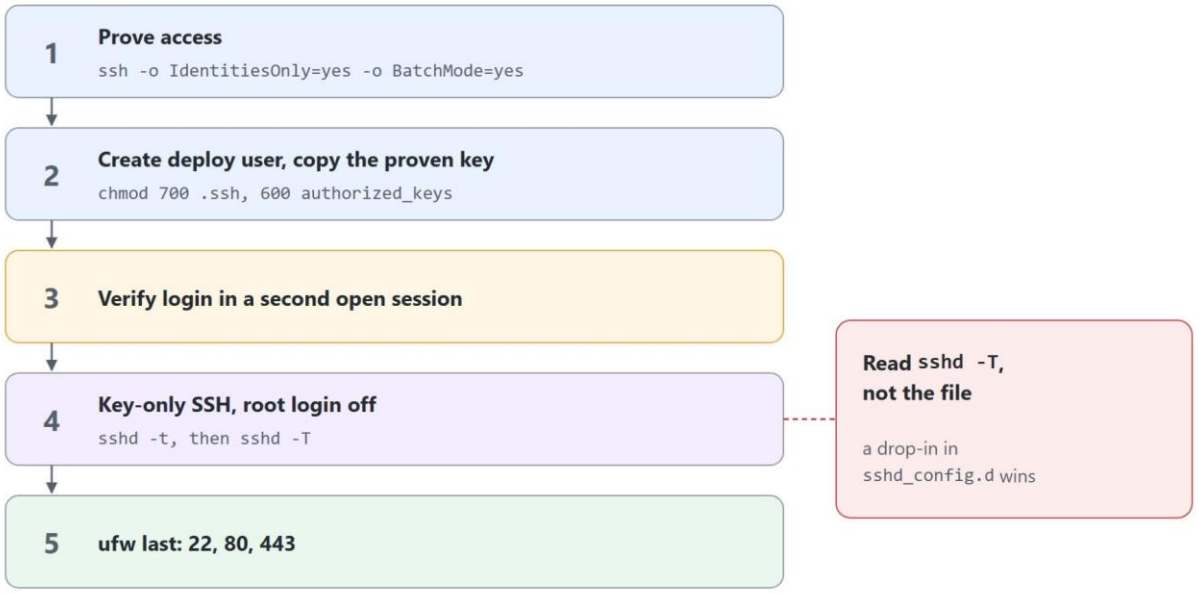
- Access, harden, runtime, app and edge, **DNS and TLS**, redeploy
- Each phase ends in a **check** against the host, not a claim
- DNS is a separate workstream that meets the deploy at **one gate**
- No preview, no reconciler, no rollback: the agent performs every guarantee
- A transcript records what was attempted; only the host records what is true



Hardening without a lockout

Prove the new door before you close the old one

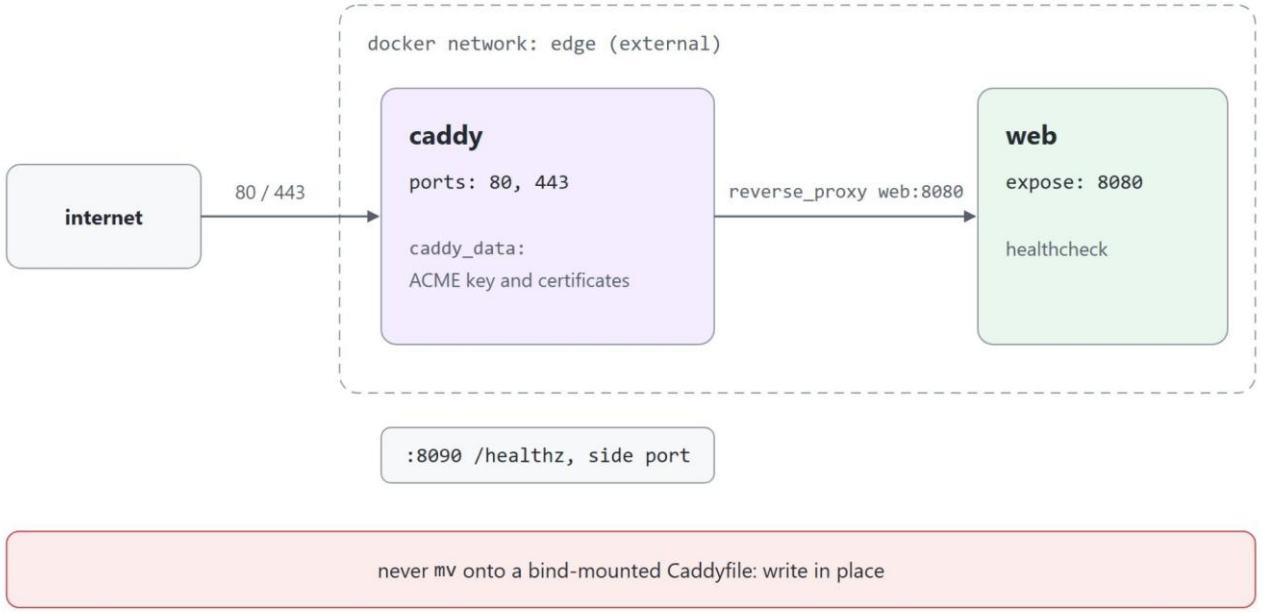
- Prove access with `IdentitiesOnly=yes` and `BatchMode=yes` first
- The **deploy** user inherits the key you already proved
- Verify its login in a second session, then turn off root and passwords
- Read the effective config with `sshd -T`, never by grepping the file
- Firewall last: allow 22, 80 and 443 before anything listens



One edge for every app

Caddy publishes 80 and 443, the apps publish nothing

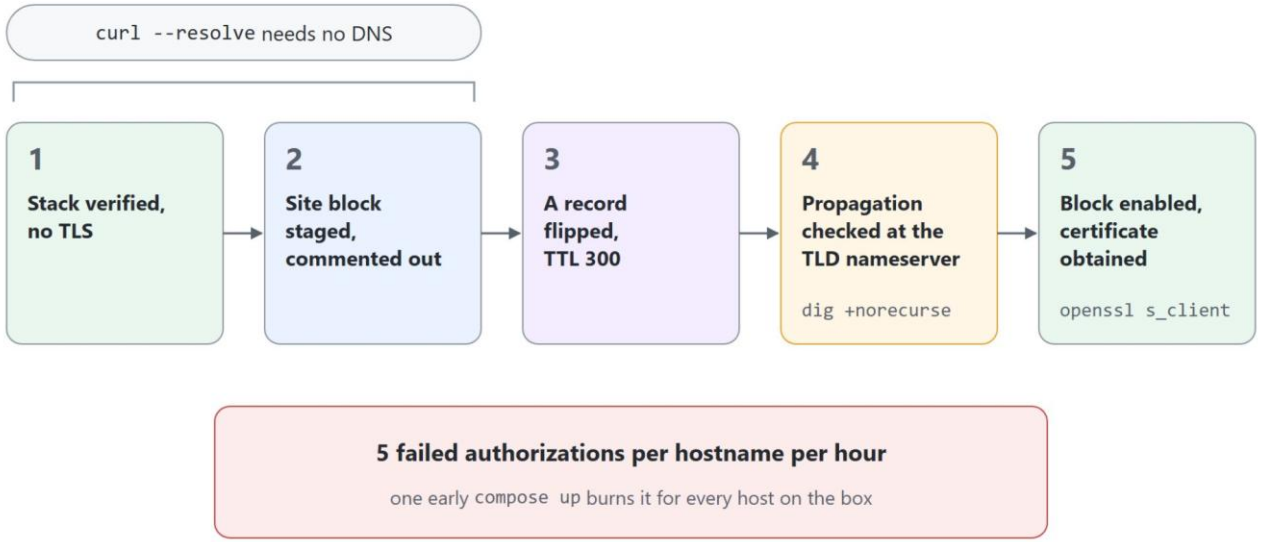
- The reverse proxy is the **only** container publishing 80 and 443
- Apps use `expose:`, not `ports:`, and join an external `edge` network
- `caddy_data` is a named volume: it holds the ACME key and certificates
- Box health probe on a **side port**, never a bare `:80` block
- Write the Caddyfile in place; `mv` leaves the old file mounted



DNS before the first certificate

Prove the edge first, flip the record, then enable TLS

- Never add a site block before the hostname **resolves** to the box
- Let's Encrypt allows **5 failed authorizations** per hostname per hour
- `curl --resolve` proves the edge live with no DNS at all
- Check propagation at the **parent zone**, not your own resolver
- Works only with `curl -k`? Report the ACME error, never fall back to self-signed



03

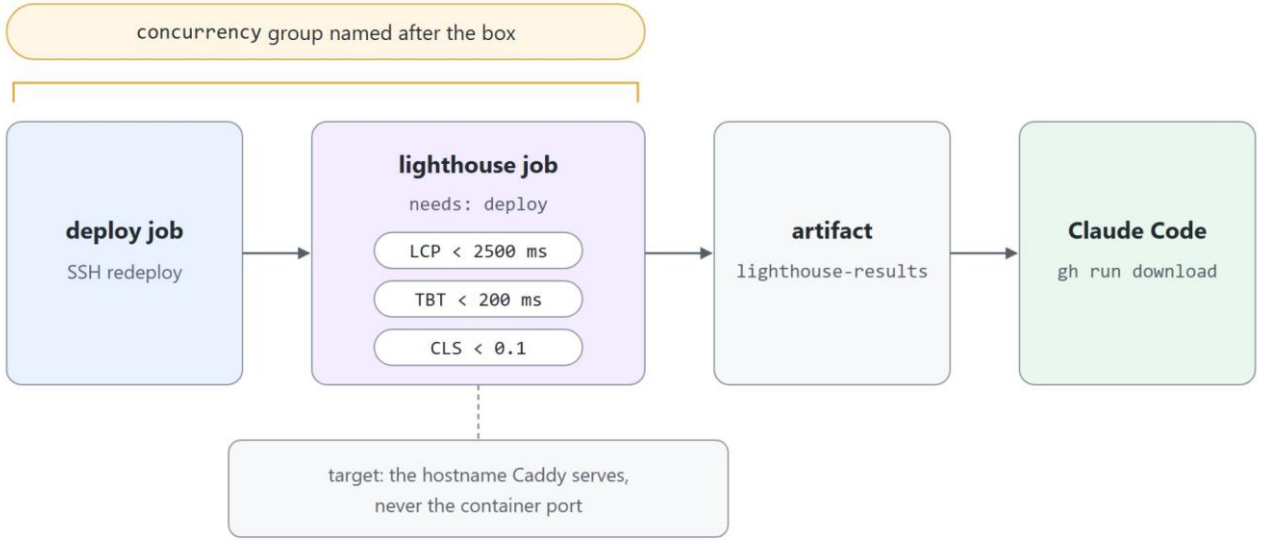
Web Vitals and SEO

Lighthouse CI and the agentic-seo fix loop

Core Web Vitals on every deploy

Lighthouse CI measures what the public sees

- Lighthouse CI runs after the deploy job via `needs:`
- Assert **LCP** under 2500 ms, **TBT** under 200 ms, **CLS** under 0.1
- Audit the public hostname, through the edge, never the container port
- The report uploads as an artifact Claude reads with `gh run download`
- One **concurrency** group per box, `cancel-in-progress: false`



Where Caddy owns the fix

Edge configuration or site source

- On your own box, part of the Lighthouse backlog is **edge configuration**
- Compression and cache headers are lines in the **Caddyfile**
- HTTP/2, HTTPS and HSTS come with automatic TLS: verify, do not add
- Images, layouts and structured data are **site source**
- Reload the edge, then re-run the audit: a valid config proves nothing



The agentic-seo fix loop

Audit, fix and verify without manual steps

- `/agentic-seo` scores **Technical SEO**, **On-Page SEO**, **Content Quality** and **Authority**
- Each audit writes a timestamped snapshot under `.seo/`
- Optimization runs read the snapshot and fix templates and front matter
- A failed Lighthouse artifact goes to Claude for root-cause analysis
- The next CI run confirms the threshold passes again



Takeaways

What to carry into your own deployments

- Give deployment its own agent with a **six-verb** allowlist
- Read every value from the **manifest**, never from memory
- Report success only with a live HTTP check and the matching image
- Harden in an order that cannot lock you out; DNS before ACME
- Measure every deploy through the edge and let the fix loop close

